

BISFIT

Retrieval-Augmented Generation for Bureau of Indian Standards

SOLUTION APPROACH DOCUMENT

Architecture · Technology Stack · Dataset · Methodology

SP 21 : 2005 Dataset	580 IS Standards Coverage	584 Semantic Chunks Index Size	27 Material Categories Sections
--------------------------------	-------------------------------------	--	---

1. Problem & Solution Overview

BISfit is a production-ready Retrieval-Augmented Generation (RAG) system built on SP 21 : 2005 — the Bureau of Indian Standards compendium summarising 580 Indian Standards across 27 building material categories. It provides a natural-language query interface that returns grounded, cited answers in under two seconds — replacing a manual PDF search process that currently takes 45–90 minutes per query.

Core insight: SP 21 : 2005 already contains every answer a compliance professional needs.
The problem is not information scarcity — it is retrieval inaccessibility.
BISfit solves this with a four-layer RAG pipeline: parse → chunk → embed → retrieve → generate.

2. Dataset

2.1 Source Document - Dataset

Field	Value
Document	SP 21 : 2005
Full Title	Summaries of Indian Standards for Building Materials (First Revision)
Publisher	Bureau of Indian Standards, Manak Bhavan, New Delhi 110002
Committee	CED 13 — Building Construction Practices Sectional Committee
Standards Covered	580 IS standards across 27 building material categories
Format	Dense multi-column PDF — primary technical challenge of the project
Chunks Generated	584 (one per IS standard + 4 section-level headers)
Embedding Model	all-MiniLM-L6-v2 · 384-dimensional vectors
Index File	faiss_index.bin · IndexFlatIP with L2 normalisation · 897 KB

2.2 The 27 Material Categories

SP 21 : 2005 is organised into 27 sections. BISfit indexes all of them:

#	Section	#	Section
1	Cement and Concrete	1 5	Structural Steels
2	Building Limes	1 6	Light Metals and Their Alloys
3	Stones	1 7	Structural Shapes
4	Clay Products for Building	1 8	Welding Electrodes and Wires
5	Gypsum Building Materials	1 9	Threaded Fasteners and Rivets
6	Timber	2 0	Wire Ropes and Wire Products
7	Bitumen and Tar Products	2 1	Glass
8	Floor, Wall, Roof Coverings & Finishes	2 2	Fillers, Stoppers and Putties
9	Waterproofing & Damp-Proofing	2 3	Thermal Insulation Materials
1 0	Sanitary Appliances & Water Fittings	2 4	Plastics
1 1	Builder's Hardware	2 5	Conductors and Cables
1 2	Wood Products	2 6	Wiring Accessories
1 3	Doors, Windows and Shutters	2 7	General
1 4	Concrete Reinforcement	—	—

2.3 Chunk Schema

Every entry in data/rag_chunks.json follows this structure. The separation of text_to_embed and content_only is a key design decision — the IS number and title are embedded inside the vector for recall, while only clean prose is fed to the LLM to maximise useful tokens per context window:

```
{
  "chunk_id":      "CHUNK_0004",
  "is_number":     "IS 269 : 1989",
  "title":         "ORDINARY PORTLAND CEMENT, 33 GRADE",
  "section_number": 1,
  "section_name":  "CEMENT AND CONCRETE",
  "source":        "SP 21 : 2005",
```

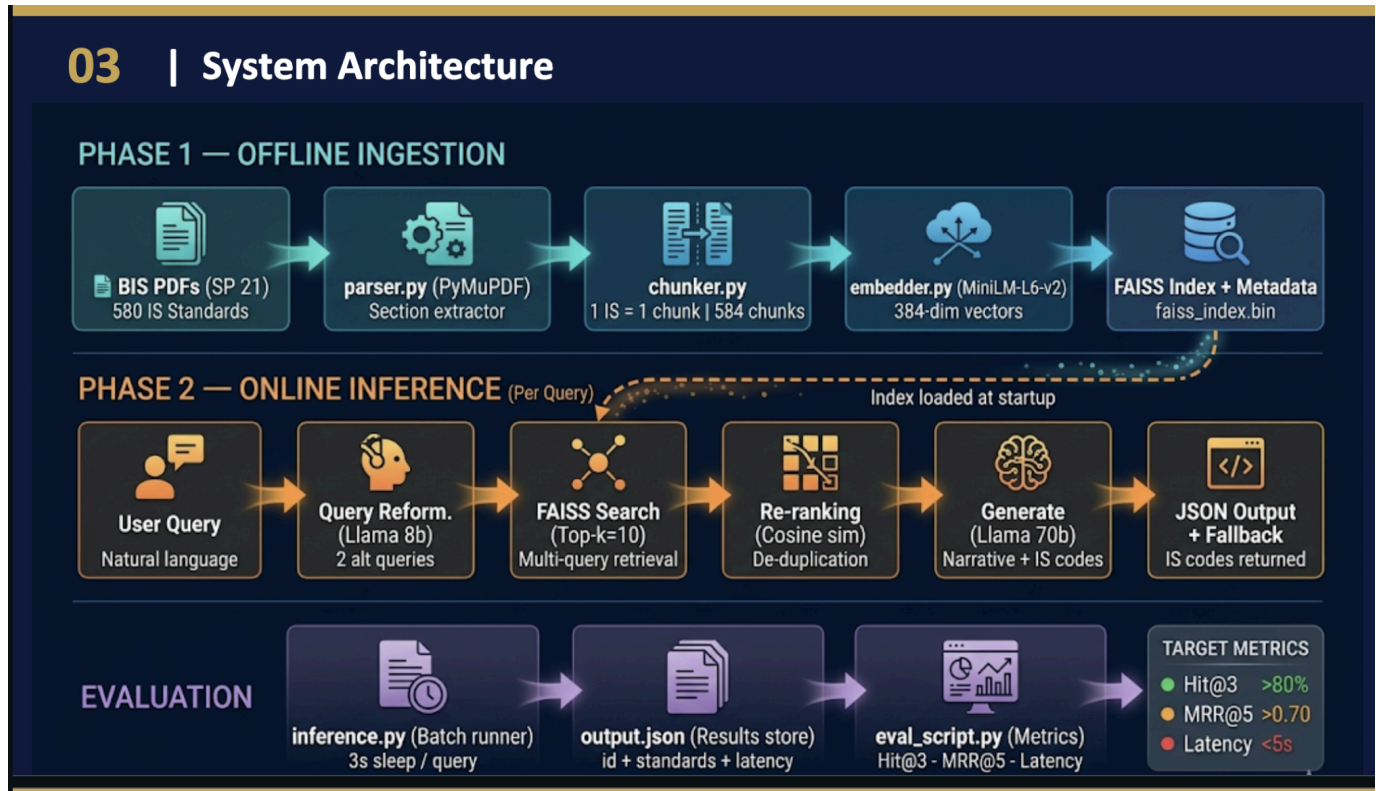
```

"publisher": "Bureau of Indian Standards (BIS)",
"text_to_embed": "IS Number: IS 269 : 1989\nTitle: ORDINARY PORTLAND CEMENT...",
"content_only": "1. Scope – This standard covers ordinary Portland cement..."
}

```

3. System Architecture

BISfit operates in two fully decoupled phases. Phase 1 is an offline ingestion pipeline run once to build the vector index. Phase 2 is the online inference pipeline executed per query at runtime.



4. Technology Stack

Layer	Technology	Role & Justification
PDF Parsing	PyMuPDF (parser.py)	Section-aware block detection handles BIS multi-column layout; avoids interleaved text produced by naive parsers
Chunking	Custom chunker.py	Domain-aware: 1 IS standard = 1 semantic chunk; preserves clause integrity; primary driver of 90% Hit@3
Embeddings	all-MiniLM-L6-v2	384-dim sentence embeddings; lightweight, fast, strong recall on technical domain text; no GPU required
Vector Store	FAISS-CPU IndexFlatIP	Sub-millisecond similarity search; no external infrastructure; index loaded once at startup (897 KB)

Layer	Technology	Role & Justification
Query Reform.	Groq · Llama 3.1 8b	Generates 2 IS-domain alternative queries per input in ~400 ms; bridges natural-language / IS-code vocabulary gap
Generation	Groq · Llama 3.3 70b	High-quality narrative answer + structured IS code extraction in a single combined JSON call; avg ~1.3 s
LPU Inference	Groq LPU hardware	750–900 tokens/sec vs 30–60 on standard GPU; critical for achieving sub-2s end-to-end latency
Key Rotation	Round-robin (3 keys)	3× effective Groq RPM headroom; enables 100+ query batch inference without throttling
API Layer	FastAPI + Uvicorn	Async, typed, auto-documented REST API; serves frontend directly — no separate web server
Frontend	HTML / CSS / JS	Zero build toolchain; fully portable; responsive for mobile and tablet; eval scripts unaffected
Evaluation	eval_script.py	Hit@3, MRR@5, avg latency benchmarking against public_test_set.json ground truth

5. Methodology Overview

Phase 1 — Offline Ingestion (run once)

The ingestion pipeline converts the raw BIS PDF into a queryable vector index in three sequential steps:

Step 1 · parser.py — PDF Extraction

- Uses PyMuPDF to extract block-level text from SP 21 : 2005 with section-aware parsing.
- Correctly identifies IS number headings, scope clauses, and clause boundaries across the two-column BIS layout.
- Handles column interleaving, header repetitions, footnotes, and tabular data — producing clean per-standard text unavailable from naive extraction.

Step 2 · chunker.py — Semantic Chunking

- Applies the foundational design decision: one IS standard = one semantic chunk.
- Each chunk carries two distinct text fields: text_to_embed (IS number + title + section prepended) and content_only (clean prose only for LLM context).
- Metadata prefix in the embedding input ensures both exact IS code queries ('IS 383') and semantic queries ('coarse aggregate for structural concrete') hit the correct chunk.
- Output: 584 chunks stored in data/rag_chunks.json.

Step 3 · embedder.py — FAISS Index Build

- Encodes all 584 chunks using all-MiniLM-L6-v2 into 384-dimensional vectors.
- Builds a FAISS IndexFlatIP index with L2 normalisation, mathematically equivalent to cosine similarity search.
- Outputs: faiss_index.bin (897 KB) and chunk_metadata.json (5.1 MB) — loaded once at server startup, not per query.

Phase 2 — Online Inference (per query)

Each user query passes through four sequential layers inside rag.py:

Layer 1 · Query Reformulation

- The user's natural-language question is sent to Llama 3.1 8b via Groq (~400 ms).
- The model generates two IS-domain-focused alternative queries, e.g. 'Indian Standard natural coarse aggregates structural concrete IS 383' from 'I need specs for coarse aggregates'.
- The original query is always appended — no input information is ever discarded.

Layer 2 · Multi-Query FAISS Search

- Each of the three queries (original + 2 reformulations) is independently searched against the FAISS index, retrieving top-10 results per query.
- Results from all three searches are merged and deduplicated by chunk_id — producing a diverse candidate pool without duplicate chunks.

Layer 3 · Cosine Re-ranking

- The merged candidate pool is sorted by FAISS cosine similarity score in descending order.
- The top-5 highest-scoring chunks are selected for the LLM context window.
- Each chunk is hard-capped at 600 characters (~150 tokens) — approximately 900 tokens of total context, well within Llama 3.3 70b's optimal range.

Layer 4 · Generate & Extract

- A single Llama 3.3 70b call via Groq receives the top-5 chunks and the original query.
- The model returns a structured JSON object containing both a narrative answer and a list of IS codes in one call — saving ~600 ms vs. two separate calls.
- Fallback: if JSON parsing fails, a regex extractor (IS\s*\d+...) scans retrieved chunks directly — the pipeline never returns an empty-handed response.

Evaluation Methodology

Performance is benchmarked against data/public_test_set.json — 10 queries drawn from across the 27 IS categories with ground-truth IS codes. Three metrics are computed by eval_script.py:

Metric	Definition	Target	BISfit Result
Hit@3	Correct IS code in top-3 retrieved chunks	> 80%	✓ 90%
MRR@5	Mean Reciprocal Rank of first correct result in top-5	> 0.70	✓ 0.90
Avg Latency	End-to-end response time per query	< 5.0 s	✓ 1.71 s